

Lab#05

Department of Computer Science (7th Semester)

Course: DevOps

Name:

Zahra Bashir

SUBMITTED TO:

Sir Waqas Saleem

Registration Number

23-BCS-092(Section-A)

DevOps Repository Link: https://github.com/zahra225/DevOps

Title: Build, Test, and Package Applications with IntelliJ IDEA and Docker

Task 1: Clone and Launch IntelliJ IDEA

I cloned the supplied IntelliJ Docker repository on my Ubuntu computer and ran setup.sh to build and start the container. I opened the browser desktop at https://localhost:3001 and launched IntelliJ IDEA until the Welcome screen appeared.

Screenshot 1: intellij_repo_clone.png



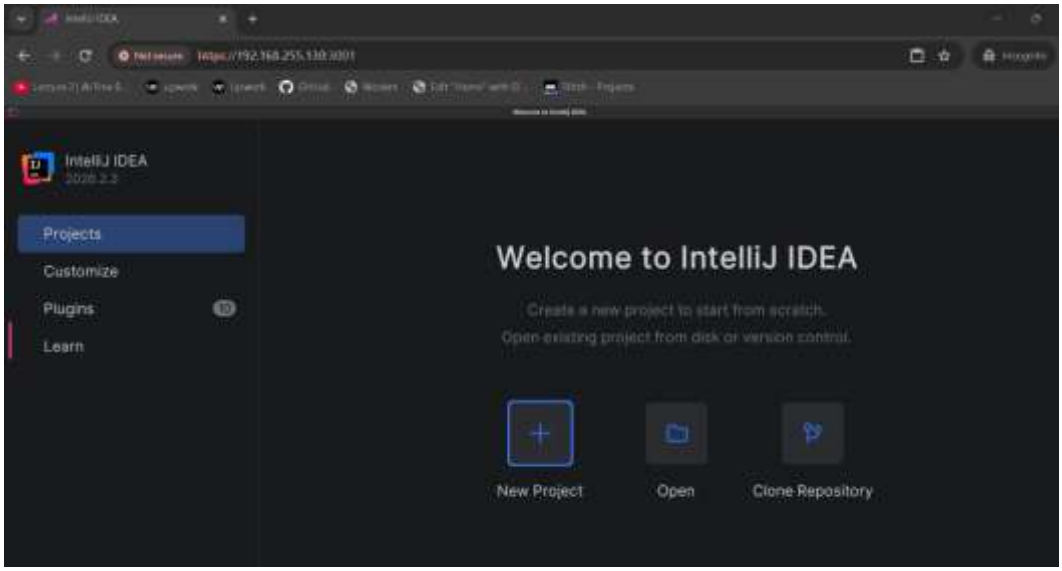
Cloning the Intellij-Idea-docker repository from the Ubuntu host terminal.

Screenshot 2: intellij_running.png



docker compose ps shows the intellij service running on port 3001.

Screenshot 3: intellij_desktop.png

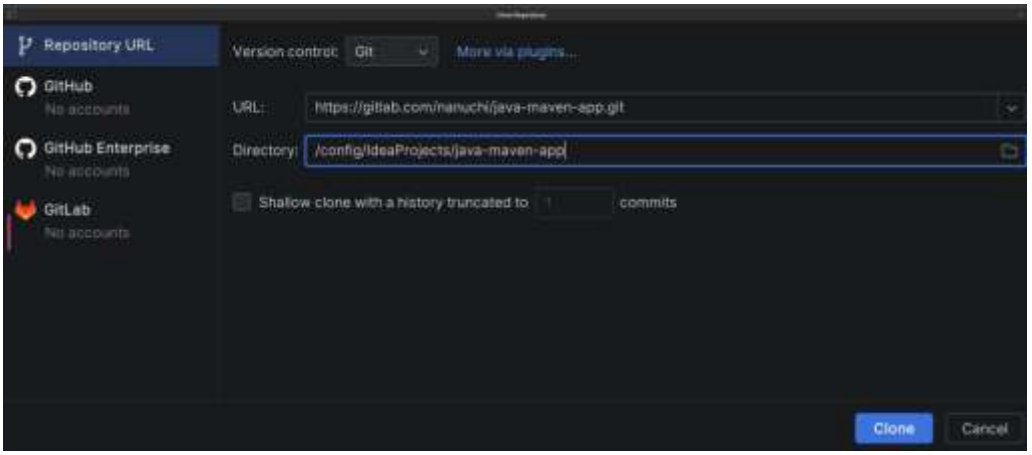


IntelliJ IDEA Welcome screen in the browser desktop, beside the Ubuntu host terminal.

Task 2: Build and Run the Maven Project

I cloned java-maven-app, installed Maven, built the project with mvn package, added the Elasticsearch 7.0.0 dependency to pom.xml and synchronized it. I packaged again, ran mvn test, found the JAR in target, ran it with java -jar and stopped it with Ctrl+C.

Screenshot 4: maven_project_clone.png



Clone dialog with the java-maven-app URL and /config/IdeaProjects/java-maven-app directory.

Screenshot 5: maven_installed.png



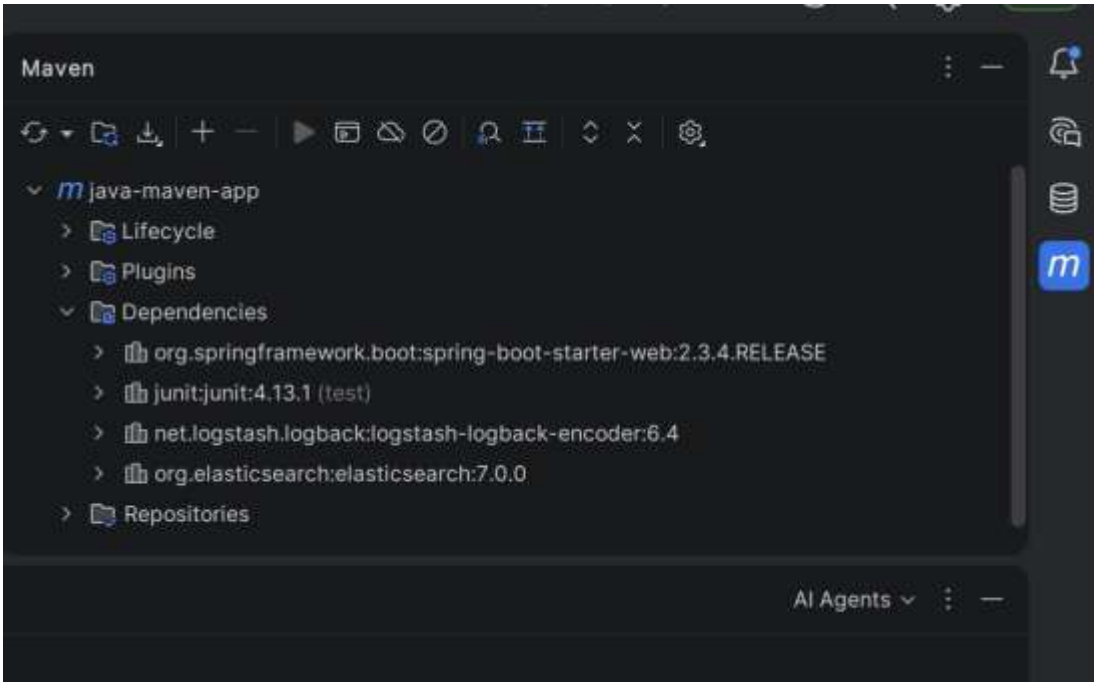
Maven installed; mvn --version output in IntelliJ's terminal.

Screenshot 6: maven_package.png



First mvn package finished with BUILD SUCCESS.

Screenshot 7: maven_elasticsearch_dependency.png



Elasticsearch 7.0.0 dependency in pom.xml and the synchronized entry in the Maven tool window.

Screenshot 8: maven_repackage.png



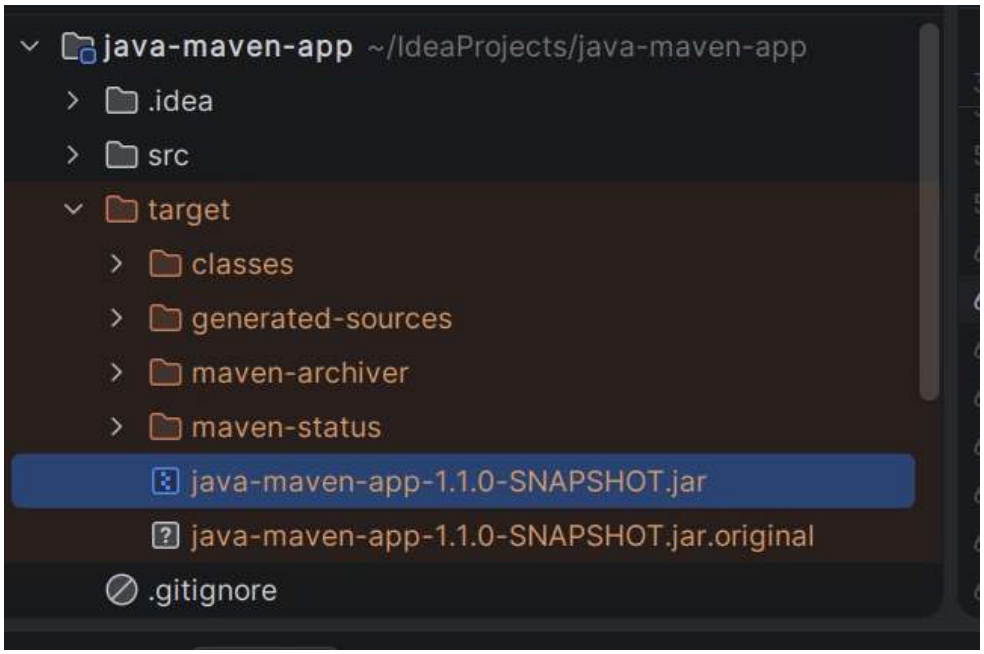
mvn package after adding the dependency finished with BUILD SUCCESS.

Screenshot 9: maven_test.png



mvn test ran the test phase; the sample has no tests, so BUILD SUCCESS with no tests to run.

Screenshot 10: maven_jar.png



target directory showing java-maven-app-1.1.0-SNAPSHOT.jar.

Screenshot 11: maven_running.png

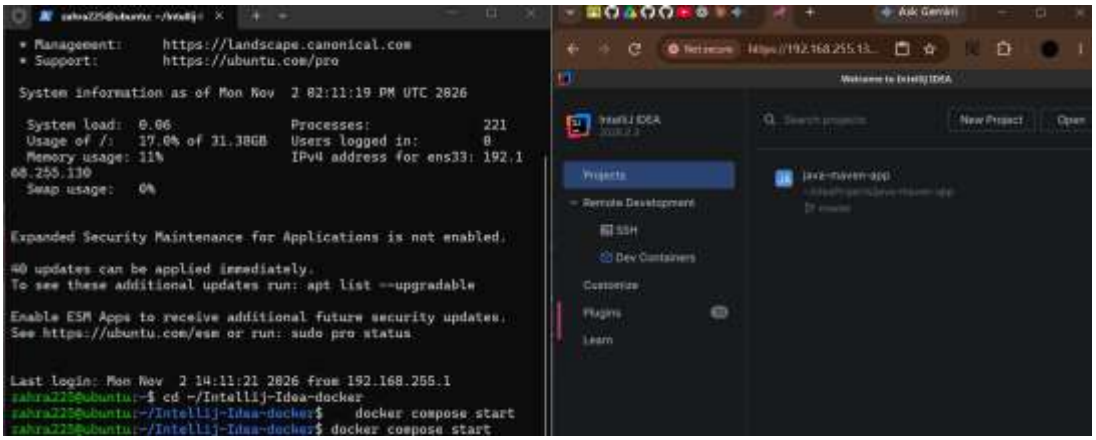


Screenshot 12: maven_stopped.png



Application stopped with Ctrl+C and the shell prompt returned.

Screenshot 13: maven_project_closed.png

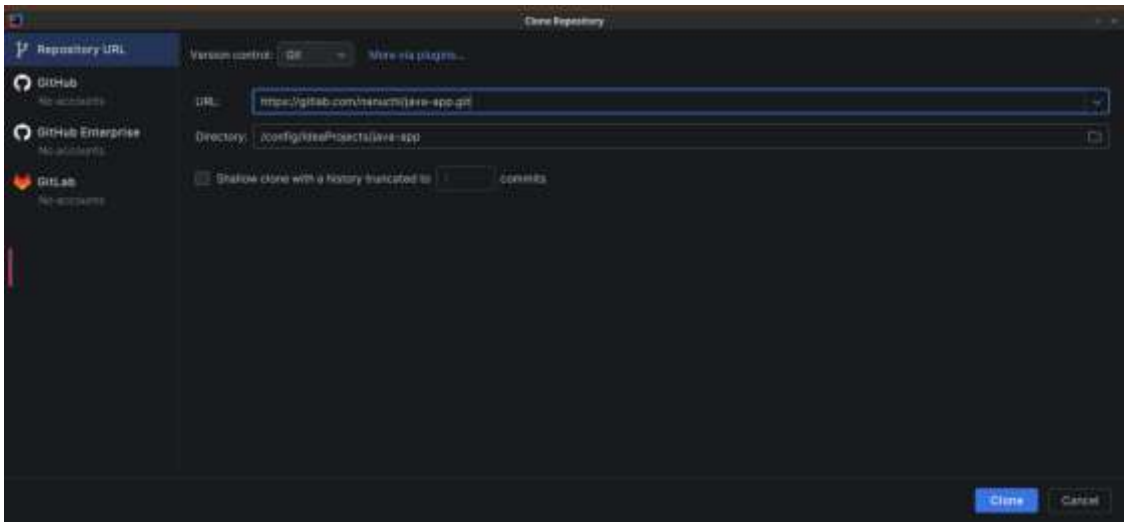


Project closed; IntelliJ Welcome screen beside the Ubuntu host terminal.

Task 3: Build and Run the Gradle Project

I cloned java-app and loaded it as a Gradle project with JDK 11 and the Gradle Wrapper. I built it with ./gradlew build, added the Elasticsearch 7.0.0 dependency to build.gradle, synchronized and rebuilt it. I then ran the generated JAR and stopped it.

Screenshot 14: gradle_project_clone.png



Clone dialog with the java-app URL and /config/IdeaProjects/java-app directory.

Screenshot 15: gradle_build.png



./gradlew build finished with BUILD SUCCESSFUL.

Screenshot 16: gradle_elasticsearch_dependency.png



Elasticsearch 7.0.0 dependency in build.gradle and the synchronized library.

Screenshot 17: gradle_rebuild.png



./gradlew build after the dependency change finished with BUILD SUCCESSFUL.

Screenshot 18: gradle_jar.png



build/libs directory showing java-app-1.0-SNAPSHOT.jar.

Screenshot 19: gradle_running.png



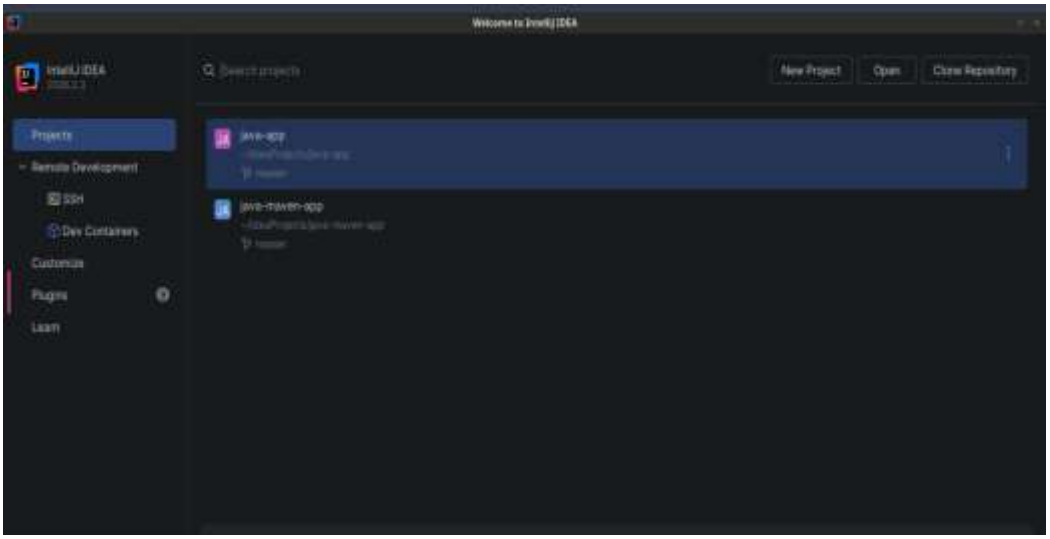
java -jar build/libs/java-app-1.0-SNAPSHOT.jar and its startup output.

Screenshot 20: gradle_stopped.png



Application stopped with Ctrl+C and the prompt returned.

Screenshot 21: gradle_project_closed.png

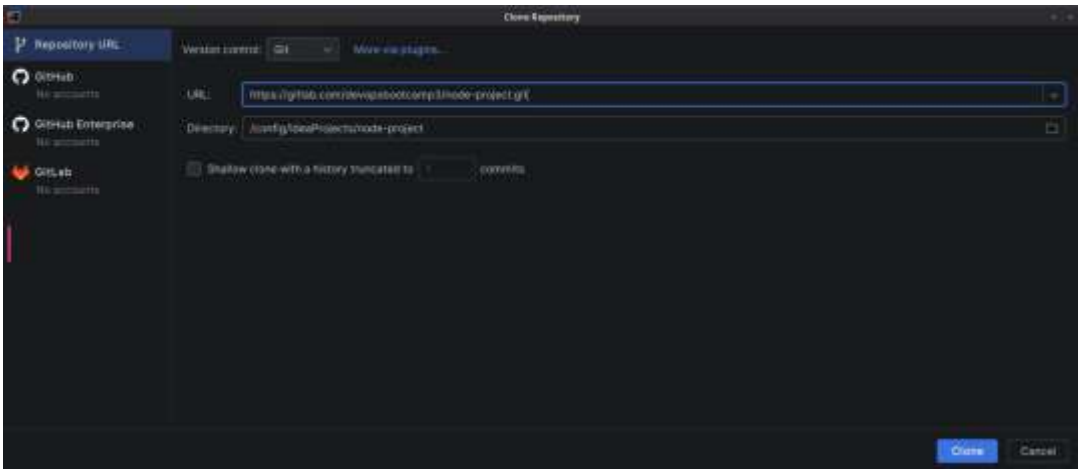


Project closed; Welcome screen beside the Ubuntu host terminal.

Task 4: Install and Run the Node.js Project

I cloned node-project, installed Node.js and npm, and installed the app dependencies with npm install. The app first failed on port 3000, so I changed server.js to port 4000 and opened it in the container's browser. I then ran npm test and created the package with npm pack.

Screenshot 22: node_project_clone.png



Clone dialog with the node-project URL and /config/IdeaProjects/node-project directory.

Screenshot 23: nodejs_installed.png



node --version and npm --version after installing Node.js.

Screenshot 24: npm_install.png



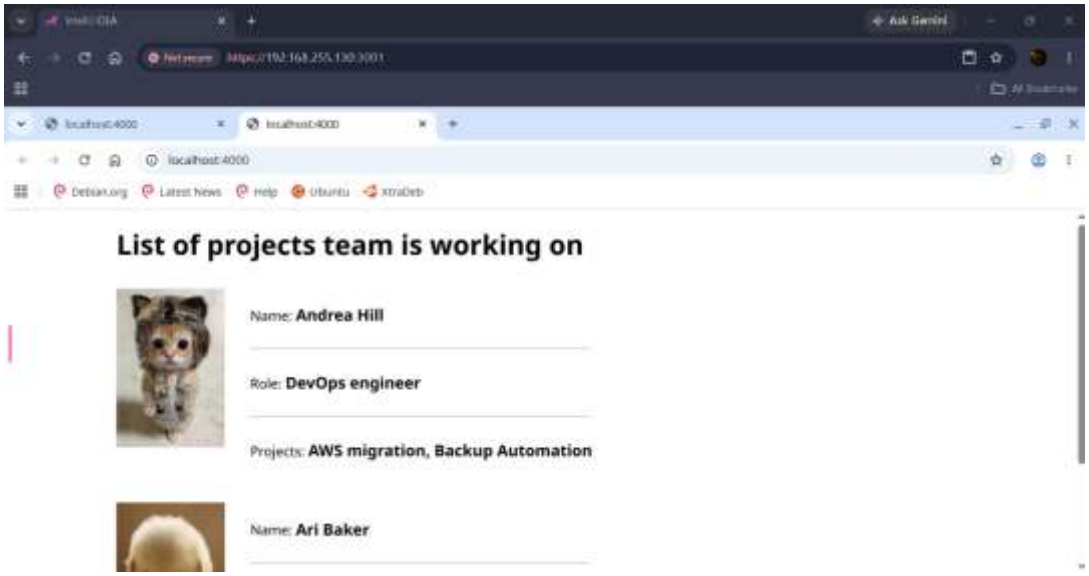
npm install completed inside the app directory.

Screenshot 25: node_running.png



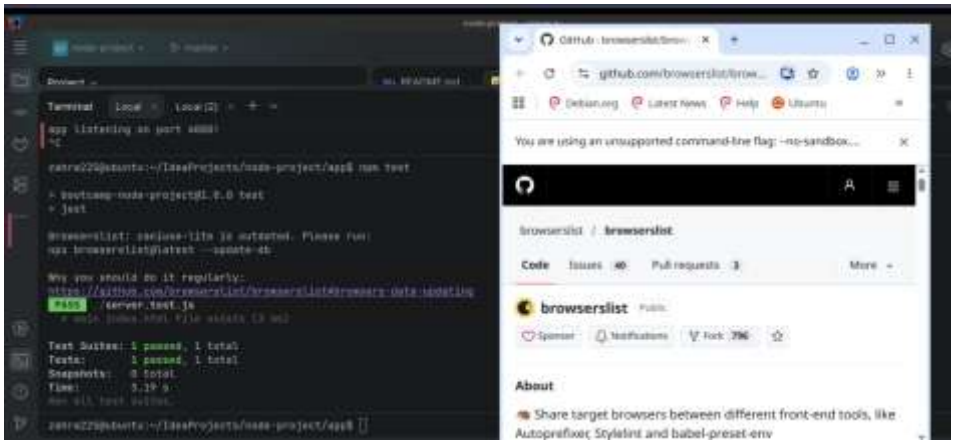
npm start printed: app listening on port 4000!

Screenshot 26: node_browser.png



http://localhost:4000 in the container's browser, with the IntelliJ terminal beside it.

Screenshot 27: npm_test.png



npm test ran Jest and the test passed.

Screenshot 28: npm_pack.png

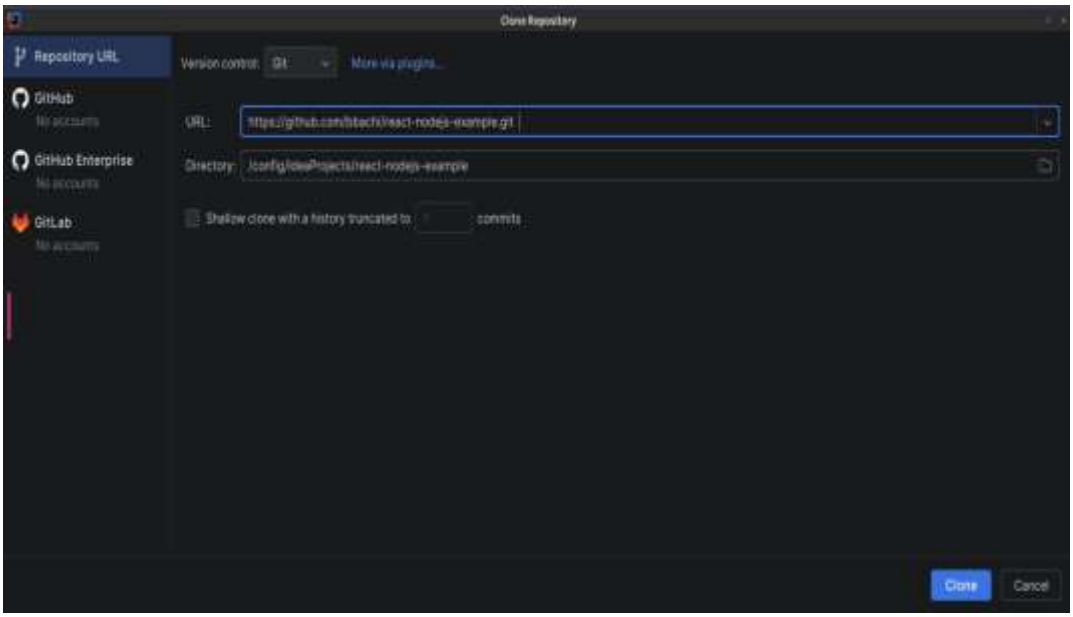


npm pack created bootcamp-node-project-1.0.0.tgz.

Task 5: Build the React and Node.js API Artifact

I cloned react-nodejs-example and installed the dependencies of its api directory. I ran npm run build, which uses webpack to create api/server.bundle.js, and viewed the bundle with cat and more.

Screenshot 29: react_node_project_clone.png



Clone dialog with the react-nodejs-example URL and directory.

Screenshot 30: react_api_dependencies.png



npm install completed and the api/node_modules directory exists.

Screenshot 31: react_api_build.png



npm run build finished and server.bundle.js was created in the api directory.

Screenshot 32: react_api_bundle.png



server.bundle.js | more showing the bundled JavaScript.

Task 6: Build a Docker Image for the Node.js Project

I reopened node-project and used its existing Dockerfile. I checked the Docker version, listed the images before the build, built the image lab5:1.0 with docker build, and listed the images again to confirm the lab5 repository and 1.0 tag.

Screenshot 33: docker_installed.png



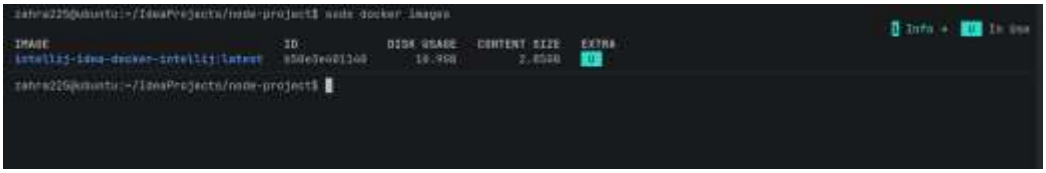
docker --version shows Docker is installed.

Screenshot 34: dockerfile_exists.png



ls shows the Dockerfile in the node-project root.

Screenshot 35: docker_images_before.png



docker images before the build (no lab5 image).

Screenshot 36: docker_build.png



docker build -t lab5:1.0 . finished successfully.

Screenshot 37: docker_images_after.png



docker images shows repository lab5 with tag 1.0 and its image ID.